CENTRO UNIVERSITÁRIO DE BRASÍLIA — CEUB

Curso de Ciência de Dados

SOUNDFLOW DATA

Planejamento Arquitetural de uma Plataforma de Engenharia de Dados para Streaming de Música

Alessandra — Matrícula: 22304883

Luiz Henrique — Matrícula: 22304604

Rafael — Matrícula: 22304454

Relatório da 1ª Avaliação — Disciplina: Engenharia de Dados

Professor: Luis C. Cardoso

Brasília, 2026

SUMÁRIO

1 Descrição do Projeto 3

1.1 Nome e contexto de negócio 3

1.2 Problema e objetivos principais 3

1.3 Principais stakeholders 4

2 Definição e Classificação dos Dados 4

2.1 Dados operacionais (batch) 4

2.2 Dados de streaming (tempo real) 5

2.3 Resumo consolidado das fontes 6

3 Domínios e Serviços 7

3.1 Identificação dos domínios de negócio 7

3.2 Serviços de cada domínio 7

3.3 Serviços compartilhados 8

3.4 Diagrama de domínios e serviços 9

4 Arquitetura e Fluxo de Dados 10

4.1 Tipo de arquitetura escolhida 10

4.2 Diagrama do fluxo ponta a ponta 11

4.3 Caminho batch 11

4.4 Caminho streaming 12

4.5 Arquitetura Medalhão — detalhamento das camadas 12

4.6 Trade-offs da arquitetura 13

5 Tecnologias — Como Será Feito 14

5.1 Ingestão 14

5.2 Armazenamento 15

5.3 Processamento e transformação 16

5.4 Orquestração 17

5.5 Consumo de dados 17

5.6 Governança, segurança e observabilidade 18

6 Considerações Finais 19

6.1 Principais riscos e limitações 19

6.2 Próximos passos para a Parte 2 20

1 DESCRIÇÃO DO PROJETO

1.1 Nome e contexto de negócio

O projeto denominado **SoundFlow Data** consiste no planejamento arquitetural de uma plataforma de engenharia de dados voltada para uma aplicação fictícia de streaming de música com modelo freemium, semelhante ao Spotify. A plataforma possui milhões de usuários que ouvem músicas, criam playlists, seguem artistas e interagem com um vasto catálogo de áudio.

O negócio depende fortemente de dados para personalização de conteúdo, descoberta musical, monetização por anúncios e retenção de assinantes premium. O projeto cobre o ciclo de vida completo dos dados, da ingestão de eventos em tempo real até a disponibilização de insights para times de negócio e para modelos de recomendação.

1.2 Problema que o projeto pretende resolver

Plataformas de streaming de música geram um volume massivo de eventos comportamentais — plays, skips, likes, buscas — em tempo real. Sem uma infraestrutura de dados adequada, surgem problemas críticos que comprometem a experiência do usuário e a viabilidade do negócio:

- Recomendações ruins: sem análise de comportamento em tempo real e histórico, o sistema não consegue personalizar o feed musical, levando ao aumento de churn.
- Invisibilidade do negócio: times de marketing, produto e financeiro não têm acesso a métricas confiáveis e atualizadas sobre engajamento, churn e receita.
- Dificuldade de escala: pipelines artesanais não suportam picos de acesso como lançamentos de álbuns ou transmissões ao vivo.
- Dados em silos: informações de catálogo, usuários, pagamentos e logs de streaming ficam isoladas em sistemas distintos sem integração.

Os objetivos principais do projeto são:

- Construir um pipeline de dados ponta a ponta integrando fontes batch e streaming.
- Disponibilizar uma camada analítica confiável para dashboards de negócio.
- Fornecer dados processados para um motor de recomendação musical.
- Garantir rastreabilidade, qualidade e governança ao longo de todo o ciclo de vida.

1.3 Principais stakeholders e usuários finais dos dados

Stakeholder	Papel	Necessidade de dados
Time de Produto	Define funcionalidades e prioridades	Métricas de engajamento, retenção e funil freemium → premium
Time de Marketing	Campanhas e aquisição	Segmentação de usuários, cohorts, taxa de churn
Time Financeiro	Controle de receita e custos	Receita por plano, royalties pagos, custo de infraestrutura
Time de Dados / ML	Modelos de recomendação	Histórico de escutas, embeddings, eventos em tempo real
Artistas / Distribuidoras	Desempenho do catálogo	Plays por faixa, região de escuta, ranking de popularidade
Engenharia de Dados	Opera e evolui a plataforma	Observabilidade dos pipelines, qualidade dos dados, SLAs

Quadro 1 — Stakeholders e necessidades de dados do SoundFlow Data.

2 DEFINIÇÃO E CLASSIFICAÇÃO DOS DADOS

O SoundFlow Data consome dados de quatro origens principais, combinando dados operacionais (batch) com dados de streaming (tempo real). A seguir, cada fonte é descrita com detalhes de origem, formato, volume estimado, frequência de atualização e latência esperada.

2.1 Dados operacionais (batch)

Dados gerados por sistemas transacionais ou atualizados em janelas de tempo definidas. São a base para análises históricas e cálculos de royalties.

2.1.1 Banco de dados de usuários

Atributo	Detalhe
Origem	PostgreSQL — serviço de autenticação e perfis
Formato	Tabelas relacionais extraídas via CDC ou dump periódico → Parquet
Volume	~5 milhões de usuários; ~200 MB por extração diária incremental
Frequência	Batch diário (00:00 UTC)
Latência	T+1h (disponível para análise até 1h após extração)
Conteúdo	ID do usuário, plano (free/premium), país, data de cadastro, preferências declaradas

Quadro 2 — Especificações da fonte de usuários.

2.1.2 Catálogo musical

Atributo	Detalhe
Origem	API REST de distribuidoras (DistroKid, TuneCore) + banco interno MySQL
Formato	JSON (via API) e CSV (dumps de distribuidoras) → Parquet
Volume	~80 milhões de faixas; ~15 GB total; ~50 MB/dia incremental
Frequência	Batch diário (com reprocessamento semanal completo)
Latência	T+2h
Conteúdo	ID da faixa, título, artista, álbum, gênero, duração, BPM, tags

Quadro 3 — Especificações da fonte de catálogo musical.

2.1.3 Dados financeiros e de assinaturas

Atributo	Detalhe
Origem	Sistema de pagamentos (Stripe API) + banco de assinaturas PostgreSQL
Formato	JSON (Stripe) → CSV → Parquet
Volume	~500 mil transações/mês; ~30 MB/mês
Frequência	Batch diário (conciliação financeira)
Latência	T+6h
Conteúdo	ID do usuário, plano, valor pago, data, status, método de pagamento

Quadro 4 — Especificações da fonte financeira.

2.2 Dados de streaming (tempo real)

Dados gerados continuamente por eventos de comportamento do usuário na plataforma. São capturados via SDK e publicados no Apache Kafka com schema Avro registrado no Confluent Schema Registry.

2.2.1 Eventos de play (escutas)

Atributo	Detalhe
Origem	SDK mobile/web da aplicação → Apache Kafka
Formato	JSON / Avro (schema registrado no Confluent Schema Registry)
Volume	~50 mil eventos/segundo no pico; ~2 bilhões de eventos/dia
Frequência	Contínuo (tempo real)
Latência	< 500 ms ate o topico Kafka
Conteudo	user_id, track_id, timestamp, device, latitude, longitude, play_duration_ms, skip_at_ms

Quadro 5 — Especificações dos eventos de play.

2.2.2 Eventos de interação (engajamento)

Atributo	Detalhe
Origem	SDK mobile/web → Kafka
Formato	JSON / Avro
Volume	~5 mil eventos/segundo

Atributo	Detalhe
Frequência	Contínuo (tempo real)
Latência	< 1 s
Conteudo	user_id, event_type (like, share, search, follow, playlist_add), timestamp, entity_id

Quadro 6 — Especificações dos eventos de engajamento.

2.2.3 Logs de infraestrutura

Atributo	Detalhe
Origem	Servidores de streaming de audio (CDN)
Formato	Texto estruturado (JSON Lines) → Kafka
Volume	~10 GB/hora
Frequência	Contínuo
Latência	< 2 s
Conteudo	server_id, user_id, track_id, bytes_transferidos, latencia_ms, codigo_http

Quadro 7 — Especificações dos logs de infraestrutura de CDN.

2.3 Resumo consolidado das fontes

Fonte	Tipo	Formato final	Volume/dia	Latência
Usuarios	Batch	Parquet	~200 MB	T+1h
Catalogo	Batch	Parquet	~50 MB	T+2h
Financeiro	Batch	Parquet	~1 MB	T+6h
Eventos de Play	Streaming	Avro/Parquet	~500 GB	< 500 ms
Engajamento	Streaming	Avro/Parquet	~50 GB	< 1 s
Logs CDN	Streaming	JSON/Parquet	~240 GB	< 2 s

Quadro 8 — Resumo consolidado das fontes de dados.

3 DOMINIOS E SERVICOS

O SoundFlow Data foi organizado em cinco domínios de negócio, seguindo princípios de baixo acoplamento e alta coesão. Cada domínio é responsável por um contexto bem delimitado e possui ownership claro sobre seus dados, comunicando-se com os demais apenas por interfaces bem definidas: eventos no Kafka ou chamadas de API.

3.1 Identificação dos domínios de negócio

Domínio	Responsabilidade principal
Catalogo	Gerencia o inventário de conteúdo musical disponível na plataforma
Usuarios	Gerencia identidade, perfil, preferências e planos dos usuários
Streaming	Captura e processa todos os eventos de comportamento em tempo real
Financeiro	Controla receita, pagamentos, repasse de royalties e conciliação
Analytics	Domínio transversal: transforma dados brutos em valor analítico e ML

Quadro 9 — Domínios de negócio do SoundFlow Data.

3.2 Servicos de cada domínio

Domínio Catalogo

Servico	Responsabilidade
catalog-ingestor	Consome dados de distribuidoras via API REST e batches diários
catalog-enricher	Enriquece metadados: gênero, BPM, mood, tags de áudio
catalog-api	Expõe o catálogo para outros domínios e para o front-end

Domínio Usuarios

Servico	Responsabilidade
user-profile-service	CRUD de perfis e preferências declaradas
auth-service	Autenticação e gerenciamento de sessões
subscription-service	Planos free/premium, upgrades e cancelamentos

Domínio Streaming / Engajamento

Servico	Responsabilidade
event-collector	SDK que captura eventos no cliente: play, skip, like
stream-processor	Consome tópicos Kafka e aplica transformações em tempo real (Flink)
engagement-aggregator	Gera agregados de curto prazo: top tracks, contagens por sessão

Domínio Financeiro / Royalties

Servico	Responsabilidade
billing-service	Integração com Stripe, gestão de cobranças
royalty-calculator	Calcula repasse por play para artistas e distribuidoras
revenue-reporter	Consolida relatórios financeiros para o time de negócios

Domínio Analytics e Recomendacao

Servico	Responsabilidade
data-warehouse-layer	Camada Gold do Lakehouse — tabelas analíticas dbt
ml-feature-store	Features prontas para modelos de recomendação
dashboard-service	Metabase / Superset conectado à camada Gold
recommendation-api	Serve sugestões personalizadas ao front-end

3.3 Servicos compartilhados (cross-domain)

Servico compartilhado	Funcao
data-quality-monitor	Valida SLAs de qualidade de dados em todas as camadas (Great Expectations)
schema-registry	Centraliza schemas Avro/Parquet (Confluent Schema Registry)
orchestrator (Airflow)	Orquestra pipelines batch de todos os domínios
observability-stack	Logs, métricas e alertas (Prometheus + Grafana)
secrets-manager	Credenciais e acessos (HashiCorp Vault)

Quadro 10 — Servicos compartilhados entre domínios.

3.4 Diagrama de domínios e serviços

O diagrama a seguir ilustra os cinco domínios de negócio, seus respectivos serviços e as dependências entre eles, além da camada de serviços compartilhados que serve a todos os domínios.

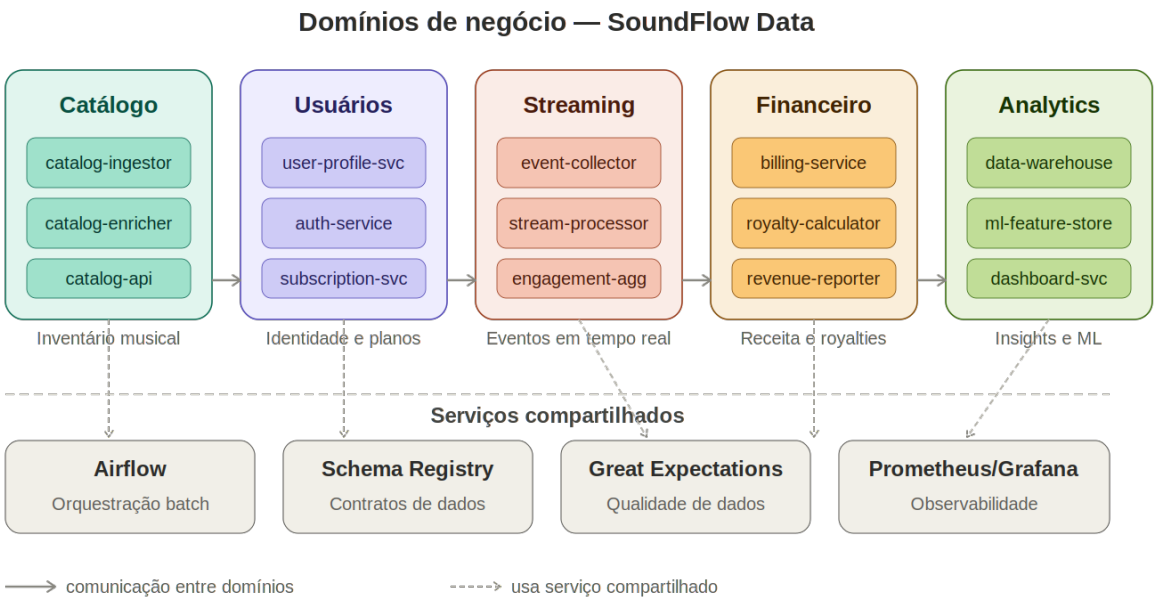

Figura 1 — Diagrama de domínios e serviços do SoundFlow Data. Fonte: autores (2026).

4 ARQUITETURA E FLUXO DE DADOS

4.1 Tipo de arquitetura escolhida

O SoundFlow Data adota uma **Arquitetura Lakehouse** com elementos do **padrão Lambda**. A escolha é justificada pelas seguintes razões:

Característica	Justificativa
Lakehouse	Unifica armazenamento de dados brutos (Data Lake) com capacidades analíticas de um Data Warehouse. Permite consultas SQL sobre dados no object storage (MinIO/S3) usando Delta Lake, eliminando a necessidade de dois sistemas separados e reduzindo custo operacional.
Padrao Lambda	A plataforma serve dois tipos de consulta: (a) análises históricas de alta precisão pelo caminho batch e (b) métricas de engajamento em quase tempo real pelo caminho streaming. O Lambda separa e otimiza cada caminho.
Arquitetura Medalhao	Dentro do Lakehouse, os dados percorrem Bronze → Silver → Gold, garantindo rastreabilidade completa, qualidade progressiva e reutilização por diferentes equipes sem duplicação de lógica.

Quadro 11 — Justificativa da arquitetura Lakehouse com padrão Lambda.

Por que não outras arquiteturas?

- **Data Warehouse puro:** insuficiente para ingestão de streams de alta velocidade e armazenamento de dados não estruturados como logs de CDN.
- **Data Lake puro:** falta suporte a transações ACID e consultas SQL eficientes — prejudicaria o time de analytics.
- **Arquitetura Kappa:** simplificaria a stack, mas o reprocessamento histórico para recálculo de royalties seria custoso sem o caminho batch dedicado.
- **Data Mesh:** adequado para organizações grandes com múltiplos times autônomos; para o escopo deste projeto, adiciona complexidade operacional desnecessária.

4.2 Diagrama do fluxo ponta a ponta

O diagrama abaixo mostra o fluxo completo dos dados da origem até o consumo, separando claramente o caminho de streaming (verde) do caminho batch (roxo). As etapas seguem a sequência: origem → ingestão →

armazenamento → transformação → consumo.

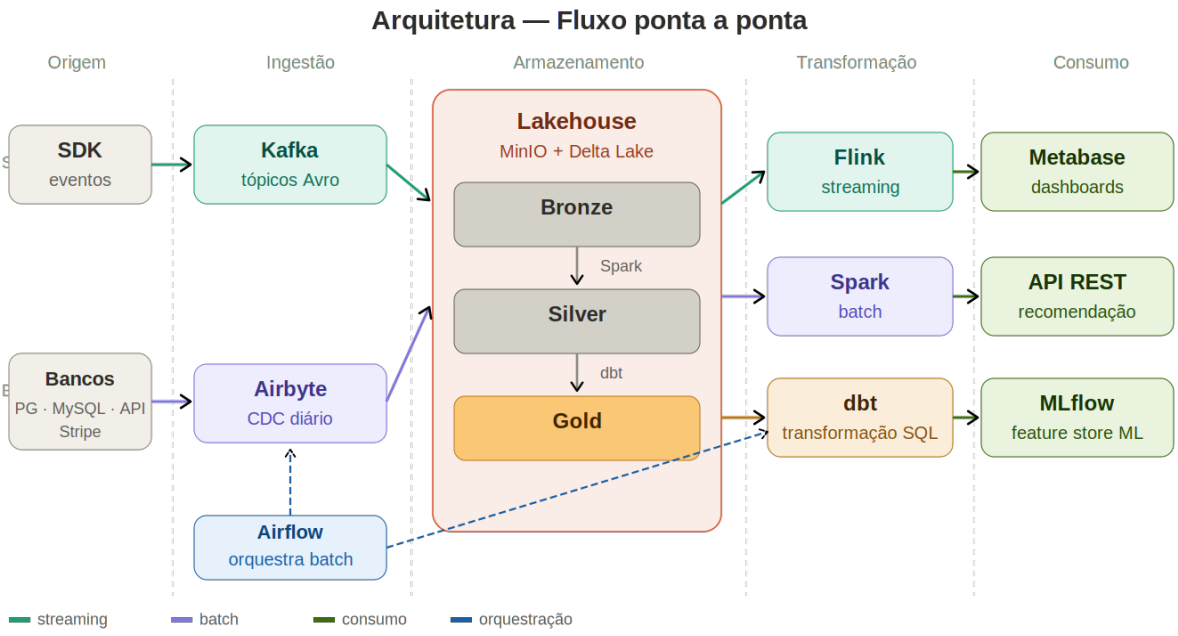

Figura 2 — Arquitetura e fluxo ponta a ponta do SoundFlow Data. Fonte: autores (2026).

4.3 Caminho batch (alta precisão)

O caminho batch processa dados operacionais extraídos dos sistemas transacionais em janelas diárias, garantindo alta precisão para análises históricas e cálculos financeiros.

O fluxo percorre: fontes batch (PostgreSQL, MySQL, Stripe) → Airbyte com CDC incremental → MinIO / Delta Lake camada Bronze (dados brutos com schema original) → Apache Spark (limpeza, deduplicação, tipagem) → camada Silver (dados limpos e padronizados) → dbt (modelos analíticos, joins, agregações) → camada Gold (tabelas de fatos e dimensões) → Metabase (dashboards de negócio) e Feature Store (features históricas para ML).

SLA: dados Gold disponíveis até 2 horas após a janela de extração.

4.4 Caminho streaming (baixa latência)

O caminho streaming processa eventos comportamentais em tempo quase real, permitindo métricas de engajamento com latência abaixo de 30 segundos.

O fluxo percorre: SDK mobile/web e logs de CDN → Apache Kafka (tópicos por tipo de evento) → Apache Flink (janelas de tempo, agregações, detecção de anomalias) → camada Bronze (eventos brutos em Parquet particionado por hora) → camada Silver (eventos enriquecidos com dados de usuário e catálogo) → tabelas de resumo de sessão (top tracks ao vivo, contagens por artista) → API de Recomendação (serve sugestões baseadas em escutas recentes).

SLA: métricas de engajamento disponíveis com latência inferior a 30 segundos.

4.5 Arquitetura Medalhão — detalhamento das camadas

Camada	Conteúdo	Formato	Quem escreve	Quem lê
Bronze	Dados brutos, sem transformação. Imutável — preserva a fonte original.	Parquet / Avro	Airbyte, Flink	Spark, Flink
Silver	Dados limpos: tipagem correta, deduplicação, nulos tratados, enriquecimento básico.	Parquet (Delta Lake)	Spark, Flink	dbt, Feature Store
Gold	Modelos analíticos finais: fatos, dimensões, métricas e agregações.	Parquet (Delta Lake)	dbt	Metabase, APIs, ML

Quadro 12 — Detalhamento das camadas da Arquitetura Medalhão.

4.6 Trade-offs da arquitetura

Acoplamento

O Kafka desacopla produtores (SDKs) de consumidores (Flink, Spark), permitindo que cada lado evolua de forma independente. O Schema Registry centraliza contratos de dados, evitando quebras silenciosas. **Risco:** o Schema Registry se torna um ponto de acoplamento — uma mudança de schema mal gerenciada pode impactar múltiplos consumidores simultaneamente.

Escalabilidade

Kafka e Flink escalam horizontalmente adicionando partições e task managers. O Lakehouse no MinIO tem escalabilidade de armazenamento praticamente ilimitada. **Risco:** o Spark tem overhead de inicialização (warm-up de JVM), o que pode ser um gargalo em jobs muito frequentes e curtos.

Disponibilidade

Kafka com replicação fator 3 garante disponibilidade de ingestão mesmo com falha de um broker. **Risco:** Airbyte e Airflow são pontos únicos de falha no caminho batch sem configuração de alta disponibilidade, o que é aceitável no escopo local deste projeto.

Confiabilidade

O Delta Lake provê transações ACID, evitando leituras parciais de dados. O Great Expectations valida a qualidade dos dados antes de promovê-los entre camadas, bloqueando dados corrompidos antes que cheguem à Gold.

Reversibilidade

A camada Bronze nunca é alterada: é o dado original. Qualquer reprocessamento parte daqui, permitindo reverter transformações com erro sem perda de informação. **Trade-off:** aumenta o custo de armazenamento progressivamente, pois os dados brutos nunca são deletados. Em produção, políticas de lifecycle comprimiriam dados com mais de 90 dias.

5 TECNOLOGIAS — COMO SERA FEITO

Para cada etapa do ciclo de vida, as tecnologias escolhidas são descritas com justificativa detalhada e explicação de como se integram às demais. Todas as escolhas priorizaram ferramentas open-source, evitando vendor lock-in e tornando o projeto viável em ambiente local.

5.1 Ingestão

Apache Kafka

Por quê: o Kafka é o padrão de mercado para ingestão de eventos em streaming de alta vazão. Suporta dezenas de milhares de eventos por segundo, oferece retenção de mensagens configurável para replay, durabilidade com replicação fator 3 e desacoplamento total entre produtores e consumidores. É a escolha natural para capturar eventos de play, skip e engajamento em tempo real.

Como se integra: o SDK mobile/web publica eventos nos tópicos Kafka (events.plays, events.engagement, logs.cdn). O Apache Flink consome esses tópicos. O Schema Registry garante que todos os eventos sigam um schema Avro validado, prevenindo dados malformados.

Alternativas descartadas: AWS Kinesis gera vendor lock-in e custo mais alto. RabbitMQ é focado em mensageria pontual sem retenção longa — em caso de falha do consumidor, os eventos seriam perdidos.

Airbyte — ingestão batch

Por quê: plataforma open-source de integração de dados com conectores nativos para PostgreSQL, MySQL e APIs REST como Stripe. Suporta CDC (Change Data Capture) para extrações incrementais, evitando recargas completas diárias. Possui interface visual para monitoramento.

Como se integra: extrai dados de PostgreSQL (usuários), MySQL (catálogo) e Stripe (financeiro) e carrega diretamente na camada Bronze do MinIO em formato Parquet.

Alternativas descartadas: Fivetran é excelente, porém pago e proprietário — inviável para um projeto acadêmico. Scripts Python customizados são flexíveis, mas sem manutenção automática de conectores e sem interface de monitoramento.

Confluent Schema Registry

Por quê: centraliza a definição de schemas Avro usados nos tópicos Kafka, garantindo compatibilidade entre versões (backward/forward). É essencial em um ambiente com múltiplos times publicando eventos: qualquer mudança no SDK é validada antes de chegar ao Flink, prevenindo quebras silenciosas.

5.2 Armazenamento

MinIO (Object Storage)

Por quê: object storage open-source 100% compatível com a API do AWS S3. Pode ser executado localmente via Docker, eliminando vendor lock-in e custo de nuvem. É o backbone do Lakehouse, armazenando todas as camadas (Bronze, Silver, Gold) em formato Parquet/Delta.

Como se integra: Airbyte e Flink escrevem diretamente no MinIO. Spark e dbt lêem e escrevem no MinIO via protocolo S3.

Alternativas descartadas: AWS S3 é ideal em produção, mas requer conta AWS e gera custos. HDFS é mais complexo de operar e o MinIO é seu substituto moderno mais leve.

Delta Lake (formato de tabela)

Por quê: o Delta Lake adiciona transações ACID, versionamento (time travel), suporte a updates/deletes e schema evolution sobre arquivos Parquet no MinIO. Isso é essencial para garantir que leituras nunca vejam dados parcialmente escritos, para permitir reprocessamento com rollback e para suportar operações de MERGE — crítico para dados de usuários que mudam de plano.

Alternativas descartadas: Apache Iceberg é tecnicamente equivalente, mas o Delta Lake tem ecossistema maior com Spark e dbt. Parquet puro não oferece ACID nem versionamento — arriscado em ambiente com escrita concorrente.

5.3 Processamento e transformação

Apache Flink — streaming

Por quê: o Flink é o motor de stream processing mais maduro do mercado open-source, com suporte nativo a janelas de tempo (tumbling, sliding, session windows), processamento de eventos fora de ordem via watermarks e semântica exactly-once com o Kafka. É ideal para calcular top tracks em janelas de 1 minuto, detectar anomalias em tempo real e enriquecer eventos com dados de usuário.

Alternativas descartadas: Spark Structured Streaming opera em mini-batches, introduzindo latência maior e menor expressividade para janelas complexas. Apache Beam é mais portável entre runners, mas adiciona complexidade desnecessária.

Apache Spark — batch

Por quê: padrão para processamento distribuído de grandes volumes de dados históricos. Oferece APIs em PySpark, suporte nativo ao Delta Lake e capacidade de processar centenas de GB com paralelismo. Utilizado para limpeza e deduplicação da Bronze para Silver, para geração de features históricas e para reprocessamento em caso de falha.

Como se integra: orquestrado pelo Airflow via SparkSubmitOperator. Lê e escreve no MinIO via Delta Lake.

dbt (Data Build Tool)

Por quê: o dbt permite escrever transformações SQL versionadas no Git, com testes automáticos de qualidade (not null, unique, referential integrity), documentação gerada automaticamente e lineage visual. É o padrão para a

camada de transformação analítica Silver para Gold. Toda a camada Gold é construída com modelos dbt.

Alternativas descartadas: transformações Spark puras são mais poderosas para dados não estruturados, mas para a Gold — que é SQL analítico — o dbt oferece organização, testes e documentação que o Spark não provê nativamente.

5.4 Orquestração

Apache Airflow

Por quê: o Airflow é o orquestrador de pipelines batch mais adotado no mercado, permitindo definir DAGs (Directed Acyclic Graphs) em Python com agendamento, retry automático, alertas e interface web. Orquestra todos os jobs Spark, modelos dbt e extrações do Airbyte no caminho batch.

Como se integra: agenda extrações do Airbyte, dispara jobs Spark via SparkSubmitOperator e executa dbt via DbtOperator. O Flink não é orquestrado pelo Airflow — opera de forma contínua e autônoma.

Alternativas descartadas: Prefect e Dagster são mais modernos e com melhor DX, porém o Airflow possui documentação mais ampla e é mais requisitado no mercado, tornando o aprendizado mais valioso.

5.5 Consumo de dados

Metabase — dashboards

Por quê: ferramenta open-source, simples de instalar via Docker, que permite criar dashboards sem necessidade de conhecimento técnico avançado — ideal para times de negócio (Marketing, Produto, Financeiro). Conecta diretamente ao Spark SQL ou a uma instância PostgreSQL com os dados da Gold.

Alternativas descartadas: Apache Superset é mais poderoso, mas mais complexo de configurar. Power BI e Tableau são ferramentas pagas e

proprietárias.

MLflow — feature store e tracking de modelos

Por quê: o MLflow gerencia o ciclo de vida de modelos de ML: rastreamento de experimentos, registro e versionamento de modelos. Utilizado para armazenar features calculadas pelo Spark e alimentar o modelo de recomendação com dados históricos e em tempo real.

5.6 Governança, segurança e observabilidade

Pilar	Ferramenta	Funcao
Qualidade de dados	Great Expectations	Valida expectativas (not null, range, uniqueness) antes de promover dados entre camadas. Bloqueia o pipeline se a qualidade estiver abaixo do limiar definido.
Seguranca / Secrets	HashiCorp Vault	Armazena credenciais de banco, API keys e tokens de forma centralizada e auditada. Nenhuma senha em código ou variável de ambiente.
Monitoramento	Prometheus + Grafana	Coleta métricas de Kafka (lag), Flink (throughput), Airflow (duração de DAGs) e MinIO (uso de disco). Grafana exibe dashboards operacionais.
Lineage / Catalogo	dbt docs + OpenMetadata	O dbt gera lineage automático das transformações SQL. O OpenMetadata cataloga todas as tabelas para discovery.
DataOps / CI	GitHub Actions	Executa testes dbt e Great Expectations em cada pull request, garantindo que novas transformações não quebrem a qualidade dos dados.

Quadro 13 — Ferramentas de governança, segurança e observabilidade.

6 CONSIDERACOES FINAIS

6.1 Principais riscos e limitações previstas

Risco	Probabilidade	Impacto	Mitigacao
Overhead de memória: Flink + Spark + Airflow + MinIO localmente consomem muita RAM	Alta	Alto	Testar com volumes reduzidos; usar Docker Compose com limites de recursos; executar Flink e Spark em modo standalone.
Complexidade do Schema Registry: erros de compatibilidade param o pipeline	Media	Alto	Definir schemas simples no início; usar compatibilidade BACKWARD; testar schemas antes de publicar.
Instabilidade do conector MySQL com CDC	Media	Medio	Usar extração full-refresh diária como fallback ao CDC.
Volume real inviável localmente	Alta	Medio	Simular streams com geradores de eventos sintéticos (Python + Faker); trabalhar com amostras.
Integração dbt + Spark complexa	Media	Medio	Usar DuckDB localmente para desenvolvimento dos modelos dbt sem precisar do cluster Spark.

Quadro 14 — Riscos identificados e estratégias de mitigação.

Limitações de escopo

- O projeto não implementará um motor de recomendação de ML completo — apenas a Feature Store e a estrutura de dados que o alimentaria.
- Os dados de produção serão simulados (gerados sinteticamente) para fins de desenvolvimento e teste.
- A plataforma será executada localmente via Docker Compose, sem alta disponibilidade real (sem replicação Kafka, sem Airflow em cluster).

6.2 Proximos passos para a Parte 2 (implementação)

A Parte 2 do projeto consistirá na implementação prática do que foi planejado neste relatório. As etapas previstas são:

Fase	Período	Atividades
Fase 1 — Infraestrutura	Semanas 1–2	Criar docker-compose.yml com Kafka, Zookeeper, Schema Registry, MinIO, PostgreSQL, Airflow e Metabase. Validar comunicação. Configurar buckets Bronze, Silver e Gold no MinIO.
Fase 2 — Ingestão	Semanas 2–3	Desenvolver gerador de eventos sintéticos (Python + Faker). Publicar eventos no Kafka com schema Avro. Configurar Airbyte para extrair usuários e catálogo de PostgreSQL local.
Fase 3 — Processamento	Semanas 3–4	Implementar job Flink consumindo events.plays para Bronze. Implementar job Spark (PySpark) para Bronze → Silver. Criar primeiros modelos dbt para a camada Gold (fct_plays, dim_users, dim_tracks).
Fase 4 — Consumo	Semanas 4–5	Conectar Metabase à camada Gold. Criar dashboard com top tracks do dia, usuários ativos e taxa de skip. Adicionar testes Great Expectations nos pipelines Spark.
Fase 5 — Orquestração	Semanas 5–6	Criar DAGs Airflow para o pipeline batch completo. Documentar o processo e gravar demonstração.

Quadro 15 — Plano de implementação para a Parte 2 do projeto.